

The RAG Chunking Bet: Why the Smart Approach Lost to the Simple Rule

The system, in plain language

We're building a RAG system over the company's documents — PDFs, mostly — with one goal: a knowledge base anyone in the company can ask questions against, instead of hunting through files.

RAG just means the model answers from our documents, not from its memory. The pipeline has five steps:

1. **Ingest** — pull the PDFs in.
2. **Chunk** — cut each document into pieces. A model can't swallow a 200-page PDF for every question, and most of any document is irrelevant to any single question, so you cut it into pieces first.
3. **Embed** — turn each piece into a vector: a long list of numbers that captures what the piece *means*. Pieces with similar meaning end up with similar numbers.
4. **Store** — put all the vectors in a vector database, which is really just a store built to answer one question fast: "which stored pieces mean something closest to this question?"
5. **Retrieve and generate** — turn the user's question into numbers the same way, pull the k closest pieces (top-k), hand those pieces to the model, and let it write the answer from them.

Chunking decides everything downstream, because the model only ever sees the retrieved chunks. Get it wrong in either direction and the answers break. Too much context and the answer drowns in irrelevant text — slower, pricier, and the model gets distracted and starts mixing facts together. Too little context and the one fact the answer needed sits in a neighboring chunk that didn't make the cut — so the model guesses.

The story

Situation. "We're building that RAG system I described — ingest, chunk, embed, store, retrieve, generate — over the company's PDFs, so anyone can ask questions against a shared knowledge base."

Task. "My call was the chunking strategy. It sounds like a detail, but it decides everything downstream. Retrieve too much context and the model drowns. Retrieve too little and it answers from thin air."

Action. "We started with the simplest thing that could work: split section by section, following the document's own structure — headings, sections, the breaks the author already put there. Then we talked ourselves into something smarter: semantic chunking, where an LLM decided the chunk boundaries. It felt like the intelligent approach. Documents are messy, sections vary wildly in length, some are grab-bags — let the model find the natural breaks.

It failed, and we caught it through human evaluation. There was no automated system. People read the chunks, then ran Q&A on top and judged whether the retrieved chunks actually produced the right responses. What we saw: answers came back either stuffed with irrelevant context or missing the piece they needed. The LLM-drawn boundaries looked sensible to a model, but they didn't line up with how real questions land on content.

We considered a hybrid — keep the rules, let the LLM refine the boundaries. Two reasons we didn't. First, we didn't have the bandwidth to run both approaches. Second, the business had a non-negotiable requirement: trace-back. For every answer the system gives, you have to point at the exact part of the exact document it came from — that's trust, and in some cases it's audit. With section-based chunks the mapping is exact: this chunk *is* section 3.2 of that document. With LLM-drawn boundaries, nobody can say why a chunk starts or ends where it does, so the trace breaks. We thought we could recover traceability on the LLM path. We couldn't.

So we went back. Not to fixed-size chunks — we'd already ruled those out, because fixed-size slicing cuts blindly: a procedure or a table split in half means neither half answers the question alone. We went back to section-by-section splitting driven by document structure. A simple rule, no model in the loop."

Result. "That fixed the retrieval behavior we'd broken. Chunks line up with the units people actually ask about, because they line up with how the documents were written. The lesson I kept: the cleverest component in the pipeline was the one we couldn't evaluate cheaply, and the boring rule won because its failure modes were visible."

What I'd do differently. "Start with the rule-based chunking, prove it, and only then earn the right to try the LLM version — and before building the LLM version, put automated measurement in place first, so we're never dependent on humans reading chunks again."

Where this story plays

- "Tell me about a time you made the wrong technical call" / a bet that failed

- "Tell me about a tough technical decision" — the tradeoff was intelligence of boundaries vs. evaluability and traceability
- Simplicity and bias for action — the boring rule beat the clever model

Follow-ups, with the honest answers

- "*How did you measure chunk quality?*" — "Human evaluation. People read the chunks, then checked whether the Q&A on top retrieved the right responses. No automated system."
- "*Why not a hybrid — rules plus LLM refinement?*" — "Two reasons. We didn't have the bandwidth to run both. And trace-back was non-negotiable — every answer has to map to an exact source section, and we couldn't recover that on the LLM path."
- "*Why not fixed-size chunks?*" — "They cut blindly. A procedure or table split across two chunks means neither chunk answers the question alone."
- "*What would you do differently?*" — "Start rule-based, prove it, then try the LLM version — with automated measurement built first, so humans never have to read chunks again."